stackmem: MCP, Tree-Sitter AST Graphs, Hybrid Containment-Trigram Scoring, and Temporal Decay for Persistent AI Coding Agent Memory

Arush Rao Lagudu
October 2026

`github.com/arushlagudu/coding-memory` · `github.com/arushlagudu/stackmem-vault`

Abstract

Every session with an LLM coding agent begins the same way: context zero. The agent that spent an hour yesterday discovering your Node version constraint, your rejected ORM, and your Supabase schema remembers none of it today, and the developer re-types the same paragraph of project context before any real work can start. We present stackmem, a four-layer memory architecture — capture, semantic organization, temporal ranking, and context delivery — that closes this loop automatically. A git post-commit hook and a Tree-Sitter AST indexer capture typed memories (*decisions*, *constraints*, *discoveries*, *rejections*) with zero manual annotation; a hybrid containment-trigram scoring function deduplicates, links, and resolves contradictions between them; an exponential-decay ranker fits the result inside a fixed token budget; and a Model Context Protocol server, CLI, and companion web dashboard deliver it back into every new session. On 48 memories drawn from stackmem's own six-week development history, the system achieves 99.4% token reduction versus full-codebase injection (36,579 → 226 tokens) and a 97.9% automatic-capture rate (47 of 48 memories saved with no human in the loop). stackmem is available today via `npm install -g stackmem`.

---

1. Introduction

Consider the following, unremarkable Tuesday. A developer opens a fresh chat with their coding agent and explains, again, that the project uses Supabase rather than Postgres directly, that Node 18 is a hard floor because of ESM import semantics, that an earlier attempt to use `dotenv` was abandoned in favor of hard-coded config, and that the webhook handler has a subtle bug around body-parser ordering that took an afternoon to track down last month. By Thursday, a second session with the same agent reintroduces the `dotenv` approach — not because the agent is unintelligent, but because it has no mechanism for remembering that this path was already tried and rejected. The developer loses the afternoon a second time. This is the *stateless agent problem*: every commercially deployed LLM coding agent today — Claude Code, Cursor, Windsurf, Codex CLI — begins each session from an empty context window, and the accumulated judgment of a project's history is either re-explained by hand or lost entirely when the chat closes.

Four existing approaches fail to solve this, each for a distinct reason:

- **Static context files (`CLAUDE.md`, `.cursorrules`).** These give an agent a fixed prelude, but the prelude is only as current as the last time a human remembered to update it. A constraint discovered in commit 4,000 is invisible unless someone manually transcribes it, and the file silently drifts out of sync with the codebase it describes.

- **Full codebase injection.** Pasting the entire repository into context is semantically complete but practically absurd: stackmem's own ten-file TypeScript source alone is 36,579 tokens (Section 4.2) — before a single line of application code is considered. For any non-trivial project this exceeds usable context budgets, and even when it fits, the agent must separately re-derive *why* any given file looks the way it does from the code alone.
- **RAG over source code.** Retrieval-augmented generation over an embedding index of the codebase retrieves what the code *is*, not what the team *decided*. A vector search for "database" will surface the Supabase client module; it will not surface the fact that a Postgres-direct approach was tried first and abandoned, because that fact was never written down as code.
- **Generic conversational memory systems (e.g., Mem0-style architectures).** These are built to remember user preferences and dialogue history for chatbots. They have no notion of a *rejected approach* as a first-class, retrievable fact — and as we argue in Section 3.2.1, this single omission is responsible for a large share of an agent's repeated mistakes.

stackmem addresses the stateless agent problem with four contributions:

1. **A four-layer memory architecture** (Capture → Semantic Organization → Temporal Ranking → Context Delivery, Section 3) that requires no manual memory curation: a git post-commit hook and a Tree-Sitter AST indexer populate the system automatically, achieving a 97.9% automated-capture rate across stackmem's own real development history (Section 4.7).
2. **A hybrid containment-trigram scoring function** (Section 3.2.2) that combines entity-set containment with character-trigram Jaccard overlap, resolving a template-collision failure mode that pure lexical entity matching cannot — e.g., distinguishing `"removed dependency: node:path"` from `"removed dependency: node:url"` — and lifting semantic-linking signal-to-noise to 7.8× over a random-pair baseline (Section 4.5).
3. **A four-type memory ontology** — *decision*, *constraint*, *discovery*, *rejection* — with the explicit inclusion of *rejection* as a first-class type, directly targeting the repeated-mistake failure mode that generic memory systems exhibit (Section 3.2.1).
4. **A temporal decay and budget-enforcement mechanism** (Section 3.3) paired with three independent delivery surfaces — an MCP server, an 18-command CLI, and a companion web dashboard, the Memory Vault (Section 3.5) — achieving 99.4% token reduction against full-codebase injection while remaining human-auditable.

---

2. Related Work

Codebase-Memory (Vogel et al., 2026). Vogel, Meyer-Eschenbach, Kohler, Grünewald, and Balzer's *Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP* [1] constructs a persistent, Tree-Sitter-derived knowledge graph of a codebase — call graphs, impact analysis, community detection — exposed to an agent via MCP, parsing 66 languages with a parallel multi-phase pipeline. Evaluated on 31 real-world repositories, it matches or exceeds a naive file-exploration agent on graph-native queries (hub detection, caller ranking) on 19 of 31 languages, at roughly ten times fewer tokens and 2.1 times fewer tool calls than exploration-by-reading.

stackmem's Layer 1 (Section 3.1) adopts the identical delivery pattern — a Tree-Sitter-derived structural index, exposed via MCP tools (`analyze_codebase`, `get_file_summary`, `find_dependencies`) — over a narrower language scope (TypeScript and JavaScript only, versus Codebase-Memory's 66). The gap we fill is orthogonal rather than competitive: Codebase-Memory's knowledge graph

answers "what does this code *do*" — its functions, classes, and call relationships. It has no node type for "why was this approach chosen" or "what was tried and abandoned." stackmem's decision/constraint/discovery/rejection layer sits one level above a structural graph like Codebase-Memory's, and the two are naturally composable: a future integration could attach stackmem memories as annotations on Codebase-Memory's graph nodes (Section 5).

Feedback-Normalized Developer Memory (Iscan, 2026). Iscan's *Feedback-Normalized Developer Memory for Reinforcement-Learning Coding Agents: A Safety-Gated MCP Architecture* [2] proposes RL Developer Memory, which treats memory selection itself as a logged, auditable contextual-decision process: an `issue_match` tool emits ranked candidates with telemetry, `issue_feedback` normalizes raw outcome labels into bounded canonical rewards, and `issue_record_resolution` retroactively links verified task resolutions back to the retrieval events that informed them. In a same-commit comparison, this architecture achieves 80.0% expected-decision accuracy and 100.0% hard-negative suppression, explicitly scoped by the authors as an auditable memory-*control* architecture rather than a general coding-agent improvement claim.

We share Iscan's commitment to auditability over opaque retrieval, and to stating narrow, directly-measured claims rather than claiming end-to-end task improvement (Section 4 measures only what stackmem can directly verify: deduplication accuracy, linking precision, token reduction — never claimed developer productivity gains). The gap is one of deployment assumption: RL Developer Memory's feedback loop requires a reward signal — some verified notion of whether a retrieved memory led to a correct resolution — which in turn requires either an RL training harness or a verifiable task outcome. stackmem targets the far more common deployment context: a stateless, chat-based coding agent with no reward model and no resolution-verification step, where memory curation must happen unsupervised, directly from git history, with no feedback signal available at all.

A-MEM (Xu et al., 2025). Xu, Liang, Mei, Gao, Tan, and Zhang's *A-MEM: Agentic Memory for LLM Agents* [3], accepted to the NeurIPS 2025 main conference track, introduces a Zettelkasten-inspired agentic memory system: new memories are written as structured notes with LLM-generated context and tags, autonomously linked to related historical memories, and — critically — can trigger *evolution* of those older memories' own attributes as the network grows. On long-term multi-hop dialogue benchmarks, A-MEM improves ROUGE-L by nearly 6× over baselines while using 1,200–2,500 tokens against 16,900 for comparable systems.

stackmem's bidirectional `memory_links` table (Section 3.2.4) directly adopts A-MEM's central idea — an interconnected knowledge network rather than a flat memory list — and our 7.8× linking signal-to-noise ratio (Section 4.5) is evidence that network is discriminative rather than promiscuous. The mechanism, however, is a deliberate simplification: where A-MEM calls an LLM per memory to generate structured notes and allows written memories to be *edited* as new context arrives, stackmem computes links with a closed-form hybrid score (Section 3.2.2) and never rewrites an existing memory's content — only resolves or supersedes it (Section 3.2.3). This is a legibility trade-off: a stackmem memory, once saved, is a fact a developer can audit and trust has not silently drifted, at the cost of A-MEM's richer, self-refining representation.

SWE-MeM (Gao et al., 2026). Gao, Zeng, Yu, Wangni, Wang, Cai, He, and Lyu's *SWE-MeM: Learning Adaptive Memory Management for Long-Horizon Coding Agents* [4] trains agents, via a memory-aware variant of GRPO, to *learn* when, what, and how to compress their own interaction history — rather than compressing on a fixed schedule — jointly optimizing compression decisions with task-resolution reward. On SWE-Bench Verified, their trained 4B and 30B models resolve

43.4% and 60.2% of issues respectively, outperforming static-workflow compression baselines.

We adopt SWE-MeM's core framing — that memory compression should be *proactive*, triggered by accumulation rather than forced reactively by overflow — in our own compression trigger (Section 3.3.3). We do not, however, adopt their mechanism: SWE-MeM's compression policy is learned against a verifiable resolve-rate reward that assumes a ground-truth notion of task success. No such signal exists for a developer's architectural decisions — there is no oracle that tells stackmem whether a `constraint` memory was "correctly" resolved. Our compression is therefore a static heuristic (a fixed unresolved-memory threshold triggering single-linkage clustering, Section 3.3.3), explicitly *not* a learned policy, and we flag closing this gap as a direction rather than a result we claim (Section 5).

AtomMem (Huo et al., 2026). Huo, Lu, Zhang, Chen, and Lin's *AtomMem: Learnable Dynamic Agentic Memory with Atomic Memory Operation* [5] decomposes high-level memory maintenance into atomic Create/Read/Update/Delete decisions, learned end-to-end via supervised fine-tuning followed by reinforcement learning; the resulting AtomMem-8B outperforms static-workflow memory baselines across three long-context benchmarks.

stackmem's `decideSaveAction` function (Section 3.2.3) is structured around exactly this atomic-CRUD decomposition — every incoming memory resolves to *create*, *duplicate* (no-op), or *supersede* (resolve-old, create-new) — a direct conceptual adoption of AtomMem's framing. The distinction is, again, learned versus rule-based: AtomMem's CRUD policy is an 8-billion-parameter model trained on exemplar trajectories; stackmem's is three deterministic thresholds (Section 3.2.2–3.2.3) that a developer can read in thirty seconds and predict exactly. We trade whatever accuracy ceiling AtomMem's learned policy might reach for full interpretability, and we report our rule's actual measured accuracy directly — F1 = 0.974 on deduplication, F1 = 0.947 on contradiction detection (Sections 4.3–4.4) — rather than relying on AtomMem's cross-benchmark numbers as a proxy for our own, structurally different, mechanism.

---

3. The stackmem Architecture

stackmem is organized as four layers, each consuming the output of the one below it:

```
=================================================
Layer 4: Context Delivery (MCP + CLI + Vault)
  session_start  |  cm start  |  stackmem-vault
=================================================
                         ^
=================================================
Layer 3: Temporal Ranking (Decay + Budget)
  exponential decay  |  2000-token greedy fill
=================================================
                         ^
=================================================
Layer 2: Semantic Organization
  typed ontology  |  hybrid scoring  |  atomic CRUD
  bidirectional linking
=================================================
```

```
                          ^
================================================
Layer 1: Capture (Git Hook + AST Indexer)
  post-commit hook  |  Tree-Sitter indexer
================================================
```

Information flows strictly upward: Layer 1 writes raw typed memories; Layer 2 deduplicates, links, and classifies them; Layer 3 ranks and budgets them for a given context window; Layer 4 renders the result into whatever surface the developer or agent is using. No layer reaches back down — a design choice that keeps each layer independently testable, and is precisely why Section 4 can benchmark deduplication accuracy, linking quality, and token efficiency as separate, non-interacting experiments.

3.1 Layer 1: Automatic Capture Pipeline

The foundation of stackmem's 97.9% automation rate (Section 4.7) is a `post-commit` git hook that runs, unattended, after every commit in a project initialized with `cm init`. The hook performs two passes over the commit: a line-level diff scan, and a commit-message quality filter.

Diff-scan capture categories. The hook inspects the diff of every commit against seven fixed templates, each mapped to a memory type:

Trigger	Example captured content	Type
New file matching `/auth|login|session|token/i`	`new auth-related file: src/session.ts`	discovery
New file matching config patterns (`.eslintrc`, `docker-compose*`, `Dockerfile`, `*.config.{ts,js}`)	`new config: next.config.ts`	constraint
New file matching `/migration/i`, `schema.prisma`, `schema.sql`	`schema change: migrations/0007_add_index.sql`	discovery
Removed `import`/`require` line	`removed dependency: node:path`	rejection
Added `package.json` dependency entry	`added dependency: zod`	decision
`CREATE TABLE` / `ALTER TABLE` / `DROP TABLE` in a schema file	`database change: CREATE TABLE memory_links (...)`	discovery
`// TODO:` / `// FIXME:` / `// HACK:` / `// XXX:` comment added	`open issue from commit: // TODO: optimize the clustering algorithm`	discovery

Commit-message capture. Independently of the diff scan, the hook evaluates the commit message itself against a quality filter — `isQualityMessage` — that rejects messages under 15 characters, bare semantic-version strings (`/^\d+\.\d+\.\d+$/`), single-character-repeated strings, and a fixed list of low-information first words (`wip`, `fix`, `update`, `temp`, `test`, ...). A message that survives the filter is saved verbatim as a `decision`. This eighth, unstructured capture path is what

allows a message like "feat: hosted Supabase backend, npm publish as stackmem, no env file required" to become a durable project memory with zero additional authoring effort — the developer's ordinary commit message *is* the memory.

The [skip ci] recursion guard. Because Layer 4 includes `cm sync`, which rewrites `.cursorrules` and `.windsurfrules` and then commits that rewrite itself (Section 3.4.2), the hook's own sync-commit would otherwise re-trigger the same post-commit hook — and because `git commit --no-verify` bypasses only `pre-commit` and `commit-msg` hooks, never `post-commit`, a naive implementation recurses indefinitely. The hook guards against this by checking its own commit message for a `[skip ci]` marker at entry and exiting immediately if present, before any diff analysis or memory-saving logic runs. This single check is the difference between one synchronization commit per real commit and an unbounded commit storm.

AST indexing. Alongside the diff scan, `cm init` and `cm analyze` invoke a Tree-Sitter-based indexer over the project's source files, currently scoped to TypeScript and JavaScript, extracting function and class declarations and import statements into a local SQLite database. This structural index is exposed through three dedicated MCP tools — `analyze_codebase`, `get_file_summary`, and `find_dependencies` — giving an agent a cheap, non-generative way to answer "what does this file export" or "what depends on this module" without a fresh full-file read.

Together, these mechanisms are why, across the 48 memories captured in stackmem's own `coding-memory` project, only one was ever saved by a human typing `cm save` directly — a 97.9% automation rate (Section 4.7).

3.2 Layer 2: Semantic Organization

3.2.1 The Typed Memory Ontology Every memory in stackmem carries exactly one of four types:

- `decision` — an architectural or implementation choice that was made, typically the commit-message capture path or an explicit `added dependency` event.
- `constraint` — a hard limit the agent must respect going forward (a Node version floor, a required environment variable, a framework choice).
- `discovery` — a learned fact about the environment that is neither a choice nor a limit (a TODO left in the code, a schema change, a new auth-related file).
- `rejection` — an approach that was tried and explicitly abandoned (a removed dependency, a reverted design).

The first three types have rough analogues in most memory systems: a fact was asserted, a limit was set, something was learned. The fourth does not, and its absence is not a minor omission. A coding agent without a durable record of *rejection* has no way to distinguish "nobody has tried this yet" from "this was tried and failed" — and will therefore periodically re-propose exactly the approaches a project has already ruled out, at the cost of the developer's time re-explaining why. By making `rejection` a first-class, equally-weighted memory type — not a note buried in a changelog — stackmem's capture pipeline turns every `removed dependency` event (Section 3.1) into a permanent, retrievable "do not re-try this" signal, with no additional developer effort.

3.2.2 Hybrid Containment-Trigram Scoring Every comparison between two pieces of memory content in stackmem — for deduplication, contradiction detection, linking, relevance scoring, and compression — is computed by a single hybrid score:

$$S_{hybrid}(a,b) = 0.7 \cdot S_{contain}(a,b) + 0.3 \cdot S_{trigram}(a,b)$$

where

$$S_{contain}(a,b) = \frac{|E(a) \cap E(b)|}{\min(|E(a)|, |E(b)|)}$$

$$S_{trigram}(a,b) = \frac{|T_3(a) \cap T_3(b)|}{|T_3(a) \cup T_3(b)|}$$

$E(x)$ is the entity set of string x: lowercased whitespace/punctuation-delimited tokens of length ≥ 5, with a small stop-word list removed. $T_3(x)$ is the character-trigram set of x: every contiguous 3-character substring of the lowercased string. $S_{contain}$ is containment rather than Jaccard similarity specifically so that a short memory whose terms are fully covered by a longer one still scores highly, rather than being penalized for the size mismatch.

Pure entity containment has a specific, diagnosable failure mode: templated content with short payloads. Consider `"removed dependency: node:path"` and `"removed dependency: node:url"` — two memories about two *different* removed packages. Under ≥ 5-character entity extraction, `node` (4 characters) and `path`/`url` (4 and 3 characters) are both filtered out as too short, leaving both strings with the identical entity set $\{\text{removed}, \text{dependency}\}$ and a containment score of 1.0 — a false duplicate. Character trigrams do not share this blind spot: the trigram sets of `"...node:path"` and `"...node:url"` diverge exactly where the strings diverge, pulling $S_{trigram}$ well below 1.0 and correcting the hybrid score accordingly. (In stackmem's actual implementation this specific collision is additionally resolved at the entity-extraction level — Section 3.2.3 — but the hybrid score is what makes the *general* class of lexical near-miss robust, not just this one template.)

3.2.3 Atomic CRUD Operations New memories are classified by `decideSaveAction`, which — following AtomMem's atomic-CRUD framing [5] — resolves every candidate to exactly one of three outcomes:

1. **Dependency-template short-circuit.** If the new content matches `/^(removed|added) dependency:/i`, its sole extracted entity is the package name itself (everything following `dependency:`), not the generic word tokens. This is evaluated *first*, before any other check, specifically to prevent the collision in Section 3.2.2 at the source and to exempt these structural event records from contradiction routing entirely — a removed-dependency fact cannot be "contradicted," only superseded by a later, unrelated add.
2. **Duplicate.** If $S_{hybrid}(\text{new}, \text{existing}) > 0.85$ and the new content contains no negation vocabulary (`switched`, `removed`, `replaced`, `no longer`, `instead`, `deprecated`, `reverted`, `changed`), the new memory is a near-duplicate and the write is silently dropped.
3. **Supersede (contradiction).** If the new content *does* contain negation vocabulary and $S_{hybrid} > 0.4$ against some existing memory, that memory is marked resolved and the new one is written in its place — the system's mechanism for "we switched from X to Y."
4. **Create.** Otherwise, the memory is written as new.

This four-step decision — template short-circuit, then duplicate, then contradiction, then default-create — is intentionally the entire algorithm: no training, no embeddings, three numeric thresholds

and one word list, all of which are printed in full above. Section 4.3–4.4 report this simple rule's measured accuracy directly.

3.2.4 Semantic Linking Whenever a memory is written, stackmem computes S_{hybrid} against every other unresolved memory in the same project and persists a bidirectional link in a `memory_links` table for every pair scoring above 0.15. Links carry the hybrid score itself, which Layer 4 surfaces to show *why* two memories are related, not merely *that* they are. Section 4.5 reports that linked pairs average a hybrid score of 0.69, against 0.09 for unlinked pairs in the same project — a 7.8× signal-to-noise ratio that indicates this 0.15 threshold is operating as a genuine discriminator, not a near-universal pass gate.

3.3 Layer 3: Temporal Ranking

3.3.1 Decay Scoring Every memory's priority for injection is governed by:

$$s(m) = (1 + \ln(1 + c_m)) \cdot e^{-\lambda t_m}$$

where c_m is the memory's access count, t_m is the number of days since its creation, and $\lambda = 0.05$.

Each term encodes a distinct design decision:

- $\ln(1 + c_m)$**, not** c_m **directly.** Access count enters logarithmically so that frequent access provides diminishing, bounded returns rather than letting one heavily-accessed memory dominate the ranking indefinitely and starve newer, equally relevant memories — a hoarding failure mode a linear term would invite.
- $e^{-\lambda t_m}$**.** Every memory decays continuously regardless of access count; nothing is permanently pinned. This exponential form is a direct computational analogue of Ebbinghaus's 1885 forgetting curve [6] — the original empirical observation that human memory retention falls off exponentially with elapsed time absent reinforcement — applied here to decide what a *coding agent* should be reminded of first.
- $\lambda = 0.05$**.** This value was chosen to produce an approximately 14-day half-life ($e^{-0.05 \times 14} \approx 0.5$), matching the cadence of a two-week developer sprint: a constraint discovered this sprint should outrank one from two sprints ago by a factor of two, all else equal, which is exactly what $\lambda = 0.05$ produces.

3.3.2 Context Budget Enforcement Memories are sorted by $s(m)$ descending and greedily added to the injected context until the next memory would exceed a fixed 2000-token budget (tokens estimated at four characters per token), at which point filling stops — the remaining lower-priority memories are omitted entirely rather than truncated mid-content. Recently added execution-log fixes (Section 3.4.1) share this same budget: memories are filled first, and whatever token allowance remains is offered to the five most recent unresolved fixes before the budget closes. This is why stackmem's own injected context measures exactly 226 tokens (Section 4.2) rather than a number tuned to look clean — it is the literal sum of whatever fit under 2000 before the loop broke.

3.3.3 Proactive Compression Once a project accumulates 20 or more unresolved memories, `cm compress` becomes available. Memories are clustered by single-linkage agglomeration over S_{hybrid} (union-find, merge threshold 0.2), and any cluster of three or more memories is collapsed into a single summary memory — the most common type in the cluster, the most frequent shared entity as

a topic label, and each member's content truncated and concatenated — with the originals marked resolved. Following SWE-MeM's framing [4], this compression is *proactive*: it is offered once a volume threshold is crossed, rather than forced reactively only when a context budget is actually exceeded. Unlike SWE-MeM, the trigger and clustering threshold are fixed constants rather than a learned policy (Section 2); we flag closing that gap explicitly in Section 5 rather than presenting the current heuristic as equivalent to a trained compression policy.

3.4 Layer 4: Context Delivery

3.4.1 MCP Integration stackmem registers an MCP server exposing eight tools: `session_start` and `session_end` for lifecycle bracketing, `save_memory` for direct writes, `search_past_solutions` for cross-project fix lookup, `record_fix` for logging problem/solution pairs, and the three AST tools from Section 3.1. `session_start` is the primary context-injection surface: it runs the full decay-rank-and-budget pipeline of Section 3.3, attaches each returned memory's linked-memory IDs (Section 3.2.4), and now additionally returns up to five open execution-log fixes, each with a `"pending"` placeholder where no solution has yet been recorded. `cm init` auto-registers this server into `~/.claude.json`, so a developer who runs one command never has to hand-edit an MCP configuration file.

3.4.2 Editor Rules Embedding For agents that read static rule files rather than speaking MCP — Cursor's `.cursorrules` and Windsurf's `.windsurfrules` — `cm sync` regenerates both files directly from the live memory store: decay-ranked, budget-capped unresolved memories grouped by type, followed by recent open fixes, with a timestamp footer. Critically, this is not a one-time scaffold: the post-commit hook (Section 3.1) invokes `cm sync` automatically after every commit, then stages and commits the regenerated rule files itself under a `[skip ci]`-marked message (guarded against recursion exactly as described in Section 3.1). The practical effect is that a Cursor or Windsurf user's project rules update themselves on every push, with no separate synchronization step the developer has to remember to run — making these two editors' memory experience fully automatic, not merely supported.

3.4.3 CLI The `cm` command line interface exposes the system's full lifecycle across 18 subcommands: `start`, `save`, `fix`, `analyze`, `search`, `resolve`, `delete`, `inspect`, `compress`, `init`, `sync`, `context`, `session-end`, `doctor`, `undo`, `export`, `import`, and `help`. `cm inspect <id>` in particular renders a single memory's full link network and related fixes as a human-readable report, and `cm doctor` verifies all five integration points (backend connectivity, MCP registration, project file, git hook, memory count) in one pass — the diagnostic surface that made it possible to validate every architectural claim in this paper directly against the running system rather than by inspection of source code alone.

3.5 The Memory Vault

Everything above runs headless: a hook, a server, a command line. The Memory Vault (`stackmem-vault.vercel.app`, a companion Next.js application in a separate repository, backed by the identical Supabase tables — `memories`, `execution_log`, `memory_links` — that the CLI writes to) is where that accumulated state becomes something a human can actually look at. The CLI writes; the Vault reads and manages.

After GitHub OAuth sign-in via Supabase Auth, a developer lands on a project dashboard listing every tracked project with its memory count and most recent activity. Each project exposes four

tabs. **Memories** groups typed memories by category with their live decay scores and access counts, with inline resolve and delete actions. **Fixes** lists `execution_log` problem/solution pairs. **Timeline** renders every memory chronologically, including resolved ones the CLI's own `cm start` deliberately omits — a complete, unfiltered history view that the token-budgeted CLI output can never show. **Graph** is a D3.js force-directed rendering of the project's `memory_links` table: nodes are memories, colored by type and sized by access count; edges are the links themselves, weighted by the exact hybrid score from Section 3.2.2.

The Graph tab is the Vault's most consequential feature, and deserves to be stated plainly: it is the first point in stackmem's entire architecture where the *implicit* knowledge network a project has been accumulating — silently, commit by commit, entirely inside Layer 2's linking logic — becomes *visible*. A developer can see, at a glance, which decisions sit at the center of a dense cluster of constraints and discoveries (and are therefore load-bearing for the project's current direction), which memories are isolated single nodes with no links at all (and are therefore candidates for being noise — a vague commit message that technically passed the quality filter but connects to nothing), and where the graph naturally partitions into separable clusters that typically correspond to the project's actual architectural boundaries. This is, concretely, an audit tool: it lets a developer answer "what does my agent actually believe about this project" by looking at a picture, rather than by reading forty lines of CLI output and reconstructing the relationships by hand. Clicking any node opens the Memory Inspector, a slide-in panel exposing that memory's full content, decay score, every linked memory's full content (not just its ID, as the CLI's `linked: <id>, <id>` line shows), and any related fixes — the same information `cm inspect` prints to a terminal, rendered for exploration rather than lookup. A global, cross-project search bar queries memories and fixes simultaneously across every project the signed-in developer has access to, and inline tooltips attached to every technical term (decay score, containment, hybrid score) are aimed specifically at developers encountering stackmem's model for the first time, who should not need to have read Section 3.2–3.3 of this paper to understand what they are looking at.

---

4. Evaluation

4.1 Dataset

All results below are measured against real production data generated by stackmem's own development process — not synthetic fixtures. Across the roughly six weeks of this project's existence, its primary tracked project (`coding-memory`) accumulated 48 memories, 47 bidirectional semantic links, and 5 `execution_log` fix records, spanning three projects in total under evaluation. Of the 48 memories, 47 were written with no human invoking `cm save` directly — they arrived exclusively through the Layer-1 capture pipeline of Section 3.1 as a side effect of ordinary commits. This dataset's defining property, and the reason we consider it a meaningful evaluation rather than a toy benchmark, is that it was never curated for the purpose of being benchmarked: every number in this section is a measurement of the system doing the job it was built to do, on itself.

4.2 Token Efficiency

We compare three approaches to giving an agent project context, holding the target project (stackmem's own `coding-memory` repository, ten TypeScript source files) fixed:

Approach	Tokens	Context coverage	Setup required
Cold session (no memory)	0	0%	none
Full codebase dump	36,579	~100% (structural)	manual, re-pasted every session
stackmem	226	25% (semantic, 1/4 key-fact categories; Section 4.6 discusses why)	`cm init`

stackmem achieves a **99.4% token reduction** relative to full-codebase injection (36,579 → 226 tokens) while remaining the only one of the three approaches that delivers *decisions and constraints* rather than raw source — the full dump provides structural completeness at the cost of every token budget a real conversation has to share with the actual task, and the cold session provides nothing at all. The 226-token figure is not a target we tuned toward; it is the literal output of the Section 3.3.2 greedy-fill loop against this project's real memory volume, and will grow, within the fixed 2000-token ceiling, as the project itself grows.

4.3 Memory Deduplication

Sampling 20 real memories and resubmitting each as if saving it again — the literal scenario the duplicate check exists to catch — against 20 unrelated cross-memory pairs as negatives, `decideSaveAction`'s duplicate classification achieves:

Metric	Value
Precision	1.00
Recall	0.95
F1	0.974

Precision of 1.00 means zero false duplicate flags across the entire sample: the rule never discarded a memory that should have been kept. The single false negative is a content-extraction limitation, not a scoring failure — memories whose entire content falls under the 5-character entity minimum (e.g., a bare `"wip"`) produce an empty entity set regardless of what they are compared against, so $S_{contain}$, and therefore S_{hybrid}, is structurally 0 even against an exact copy of itself. This is a known, diagnosed boundary condition of the entity-extraction step (Section 3.2.2), not a gap in the CRUD logic built on top of it.

4.4 Contradiction Detection

Synthetic contradictions were generated by prepending `"switched from "` or `"no longer using "` to real memory content and resubmitting against the original, evaluated against the same 20 unrelated pairs as negatives (restricted, in both the positive and negative sample, to genuine decision/discovery-style content — Section 3.2.3's dependency-template short-circuit deliberately exempts structural event records like `removed dependency: X` from ever being treated as contradictable, so including them here would measure whether that exemption holds rather than whether contradiction detection works):

Metric	Value
Precision	1.00
Recall	0.90
F1	0.947

As in Section 4.3, precision is perfect — no unrelated memory pair was ever misclassified as a contradiction — and the two false negatives are the same zero-entity boundary case (`"switched from wip"` has no more extractable entities than `"wip"` alone). Both failure modes in Sections 4.3 and 4.4, notably, are the *same* underlying cause, which is a more useful finding than two unrelated 90-95% numbers would be: the entity-extraction minimum length is the single lever that would move both results, and future work (Section 5) should treat it as such rather than tuning the two CRUD thresholds independently.

4.5 Semantic Linking Quality

Across the 47 real links in the dataset, the average hybrid score of a *linked* pair is 0.69; the average hybrid score of a same-project pair that is *not* linked, computed exhaustively rather than sampled, is 0.09 — a **7.8× signal-to-noise ratio**. This is, concretely, the claim that the 0.15 linking threshold (Section 3.2.4) is discriminative rather than promiscuous: a random pair of memories from the same project very rarely crosses it, while an actually-linked pair sits, on average, more than four and a half times above it. We note as an honest ablation that prior to the hybrid-score and dependency-template fixes described in Section 3.2.2–3.2.3, this same measurement — on pure entity containment, before the trigram component and package-name entity extraction were introduced — produced a signal-to-noise ratio of only 1.7×, driven almost entirely by exactly the template-collision failure mode Section 3.2.2 describes. The 7.8× figure is not an incidental property of this scoring function; it is the direct, measured consequence of the two fixes this paper describes.

4.6 Context Relevance

Against five realistic maintenance tasks ("fixing git hook to prevent recursion," "adding Supabase database storage," etc.) scored with `scoreMemoryRelevance` over the full memory pool, we measure Precision@1 = 0.60 and Precision@5 = 0.12. We report this honestly rather than selectively: with only 13 live memories in the primary evaluated project at the time of this measurement, most individual tasks have at most one genuinely relevant memory to find at all, which places a hard ceiling on Precision@5 that has nothing to do with ranking quality — three or four of the five slots are necessarily filled by the next-best-available memory regardless of how good the scorer is, because nothing better exists in the corpus. Precision@1, by contrast, is 0.60 — the top-ranked result is the correct one three times out of five — and we consider Precision@1 the operationally relevant number here, since a developer or agent consulting `cm context <task>` reads the first result first and is far more sensitive to whether *that* result is right than to the composition of positions two through five. We expect Precision@5 to improve as a direct function of corpus size rather than requiring any change to the scoring function itself, and flag the 48-memory corpus as a limitation in its own right (Section 5).

4.7 Automation Rate

Of the 48 memories in the primary evaluated project, 47 — **97.9%** — were captured automatically by the Layer-1 pipeline (Section 3.1) with no human invoking `cm save`. Exactly one was a manual

save. This is, in a direct sense, the paper's central empirical claim: a system whose core promise is that developers should not have to manually curate their own agent's memory, demonstrated on itself, with the git hook rather than any human developer as the primary value driver of the entire dataset this evaluation is built on.

---

5. Limitations and Future Work

We report five specific limitations rather than a general caveat, each framed as a concrete direction:

1. **Lexical scoring, not embeddings.** Every scoring function in Section 3.2 is lexical — entity overlap and character trigrams — with no learned semantic representation. Two memories that express the same fact in entirely different vocabulary (`"we moved persistence to Supabase"` vs. `"database layer now backed by Postgres-as-a-service"`) will score poorly under S_{hybrid} despite being near-duplicates in meaning. An embedding-based similarity component, blended into the existing hybrid formula as a third term, is the direct extension.
2. **A 48-memory benchmark corpus.** Every result in Section 4 is measured against one real but modestly sized project. The Precision@5 ceiling discussed in Section 4.6 is a direct symptom of corpus size, and deduplication/contradiction accuracy (Sections 4.3–4.4) should be re-validated as the corpus grows by one to two orders of magnitude, ideally across multiple independent codebases rather than stackmem's own.
3. **Device-scoped, not account-scoped, identity.** Memories are currently partitioned by a per-device identifier rather than a user account, which means a developer's memory does not currently follow them across machines, and a team cannot yet share a project's memory pool as a single coherent entity — only the Vault's OAuth-gated read access (Section 3.5) crosses that boundary today, and only for reading, not for writing.
4. **No adversarial evaluation of the CRUD classifier.** Sections 4.3–4.4 evaluate `decideSaveAction` (Section 3.2.3) on naturally-occurring and lightly-synthesized content. We have not evaluated it against adversarially constructed near-duplicates or contradictions specifically designed to evade the negation-word list or the 0.85/0.4 thresholds, which a determined attempt to game the memory store — or simply unusual real phrasing — could plausibly find.
5. **A static, not learned, compression and CRUD policy.** As discussed in Section 2, both AtomMem [5] and SWE-MeM [4] demonstrate that these specific decisions — when to compress, how to classify a candidate memory — can be learned and can outperform static workflows. stackmem's current implementation is deliberately the static baseline those papers compare against, chosen for interpretability (Section 3.2.3); closing this gap without sacrificing that interpretability is open.

Beyond directly addressing these five points, we see three further directions: cross-device account sync enabling genuine team-shared memory pools (directly motivated by Limitation 3); tighter integration between stackmem's decision layer and structural knowledge graphs in the style of Codebase-Memory [1], attaching memories as annotations on graph nodes rather than keeping the two systems disjoint; and broader MCP-ecosystem integration testing beyond the Claude Code / Cursor / Windsurf surfaces this paper evaluates, since the Model Context Protocol itself is designed to be agent-agnostic and stackmem's architecture makes no Claude-specific assumption at any layer.

---

6. Conclusion

The stateless agent problem is not a minor inconvenience — it is a recurring tax on every developer who uses an LLM coding agent across more than one session, paid every single time in re-explained context and, worse, in repeated mistakes an agent has no mechanism to remember it already made. This paper presented stackmem, a four-layer memory architecture — automatic capture via git hooks and AST indexing, semantic organization via a typed ontology and hybrid containment-trigram scoring, temporal ranking via exponential decay and hard budget enforcement, and context delivery via MCP, an 18-command CLI, and the Memory Vault web dashboard — built specifically to eliminate that tax without asking a developer to maintain anything by hand.

Measured against stackmem's own real development history, the system achieves 99.4% token reduction over full-codebase injection (36,579 $\rightarrow$ 226 tokens), F1 = 0.974 on memory deduplication, F1 = 0.947 on contradiction detection, a 7.8$\times$ semantic-linking signal-to-noise ratio, and — the number we consider most consequential — a 97.9% fully-automated capture rate, with only one memory in the entire evaluated dataset ever requiring a human to type `cm save`.

stackmem is available at `https://github.com/arushlagudu/coding-memory` and installable via `npm install -g stackmem`.

References

[1] M. Vogel, F. Meyer-Eschenbach, S. Kohler, E. Grünewald, and F. Balzer. *Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP.* arXiv:2603.27277, 2026.

[2] M. Iscan. *Feedback-Normalized Developer Memory for Reinforcement-Learning Coding Agents: A Safety-Gated MCP Architecture.* arXiv:2605.01567, 2026.

[3] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. *A-MEM: Agentic Memory for LLM Agents.* arXiv:2502.12110. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, Main Conference Track, 2025.

[4] S. Gao, W. Zeng, Z. Yu, J. Wangni, C. Wang, K. Cai, S. He, and M. R. Lyu. *SWE-MeM: Learning Adaptive Memory Management for Long-Horizon Coding Agents.* arXiv:2606.28434, 2026.

[5] Y. Huo, Y. Lu, Z. Zhang, H. Chen, and Y. Lin. *AtomMem: Learnable Dynamic Agentic Memory with Atomic Memory Operation.* arXiv:2601.08323, 2026.

[6] H. Ebbinghaus. *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie.* Duncker & Humblot, Leipzig, 1885.